

SIMATS ENGINEERING

ASSESSMENT TOOL 1 (High)- Analytical Problem Solving

COURSE NAME: Data Structures

COURSE CODE:CSA03

COURSE OUTCOME COVERED

C02 : Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques. (BTL3, BTL4)

Assessment Tool Weightage: **30%**

QUESTIONS:

1. Consider a linked list representing the sales of different products in a store for a week. Each node of the linked list contains the product ID and the number of units sold on a specific day. Implement a linked list data structure and provide functions to: Insert new sales data for a given product on a specific day. Calculate the total sales for each product at the end of the week. Find the product with the highest sales and its corresponding day. Display the sales data for a specific product for the entire week. Use your own data for product.

CODE:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node
```

```
{
```

```
    int productID;
```

```
    int day;
```

```
    int unitsSold;
```

```
    struct Node *next;
```

```
};
```

```

struct Node *head = NULL;

// Insert sales data
void insert(int pid, int day, int units)
{
    struct Node *newNode, *temp;

    newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->productID = pid;
    newNode->day = day;
    newNode->unitsSold = units;
    newNode->next = NULL;

    if (head == NULL)
    {
        head = newNode;
    }
    else
    {
        temp = head;
        while (temp->next != NULL)
        {
            temp = temp->next;
        }
        temp->next = newNode;
    }
}

```

```

// Display all sales records
void display()
{
    struct Node *temp = head;

    printf("\nSales Records:\n");
    while (temp != NULL)
    {
        printf("Product ID: %d Day: %d Units Sold: %d\n",
            temp->productID, temp->day, temp->unitsSold);
        temp = temp->next;
    }
}

// Calculate total sales for each product
void totalSales()
{
    struct Node *temp = head, *t;
    int checked[100], count = 0, i, total, found;

    printf("\nTotal Sales of Each Product:\n");

    while (temp != NULL)
    {
        found = 0;

        for (i = 0; i < count; i++)

```

```

    {
        if (checked[i] == temp->productID)
        {
            found = 1;
            break;
        }
    }

    if (!found)
    {
        total = 0;
        t = head;

        while (t != NULL)
        {
            if (t->productID == temp->productID)
                total += t->unitsSold;

            t = t->next;
        }

        printf("Product %d = %d units\n", temp->productID, total);
        checked[count++] = temp->productID;
    }

    temp = temp->next;
}
}

```

```
// Find product with highest sales
```

```
void highestSales()
```

```
{
```

```
    struct Node *temp = head;
```

```
    int max, pid, day;
```

```
    if (head == NULL)
```

```
    {
```

```
        printf("List is Empty.\n");
```

```
        return;
```

```
    }
```

```
    max = temp->unitsSold;
```

```
    pid = temp->productID;
```

```
    day = temp->day;
```

```
    while (temp != NULL)
```

```
    {
```

```
        if (temp->unitsSold > max)
```

```
        {
```

```
            max = temp->unitsSold;
```

```
            pid = temp->productID;
```

```
            day = temp->day;
```

```
        }
```

```
        temp = temp->next;
```

```
    }
```

```
printf("\nHighest Sales:\n");  
printf("Product ID: %d\n", pid);  
printf("Day: %d\n", day);  
printf("Units Sold: %d\n", max);  
}
```

```
// Display sales for a specific product
```

```
void productSales(int pid)
```

```
{
```

```
    struct Node *temp = head;
```

```
    int found = 0;
```

```
    printf("\nSales of Product %d:\n", pid);
```

```
    while (temp != NULL)
```

```
    {
```

```
        if (temp->productID == pid)
```

```
        {
```

```
            printf("Day %d : %d units\n", temp->day, temp->unitsSold);
```

```
            found = 1;
```

```
        }
```

```
        temp = temp->next;
```

```
    }
```

```
    if (!found)
```

```
        printf("Product not found.\n");
```

```
}
```

```
int main()
```

```
{
```

```
    int n, i, pid, day, units, searchID;
```

```
    printf("Enter number of sales records: ");
```

```
    scanf("%d", &n);
```

```
    for (i = 1; i <= n; i++)
```

```
    {
```

```
        printf("\nRecord %d\n", i);
```

```
        printf("Enter Product ID: ");
```

```
        scanf("%d", &pid);
```

```
        printf("Enter Day (1-7): ");
```

```
        scanf("%d", &day);
```

```
        printf("Enter Units Sold: ");
```

```
        scanf("%d", &units);
```

```
        insert(pid, day, units);
```

```
    }
```

```
    display();
```

```
totalSales();
```

```
highestSales();
```

```
printf("\nEnter Product ID to display weekly sales: ");
```

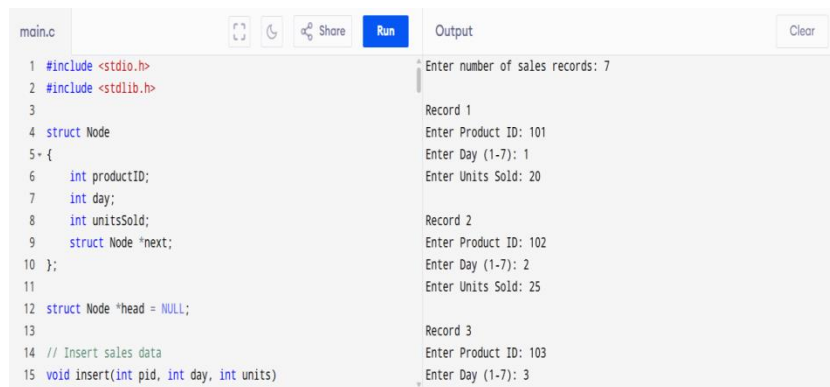
```
scanf("%d", &searchID);
```

```
productSales(searchID);
```

```
return 0;
```

```
}
```

INPUT&OUTPUT:



The screenshot shows a code editor with a file named 'main.c'. The code defines a linked list structure and a function to insert sales data. The output window shows the program's execution, starting with a prompt for the number of sales records (7). It then displays three records, each with prompts for Product ID, Day (1-7), and Units Sold.

```
main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node
5 {
6     int productID;
7     int day;
8     int unitsSold;
9     struct Node *next;
10 };
11
12 struct Node *head = NULL;
13
14 // Insert sales data
15 void insert(int pid, int day, int units)
```

Output

Enter number of sales records: 7

Record 1

Enter Product ID: 101

Enter Day (1-7): 1

Enter Units Sold: 20

Record 2

Enter Product ID: 102

Enter Day (1-7): 2

Enter Units Sold: 25

Record 3

Enter Product ID: 103

Enter Day (1-7): 3

```
Record 3
Enter Product ID: 103
Enter Day (1-7): 3
Enter Units Sold: 30
```

```
Record 4
Enter Product ID: 104
Enter Day (1-7): 4
Enter Units Sold: 35
```

```
Record 5
Enter Product ID: 105
Enter Day (1-7): 5
Enter Units Sold: 40
```


Record 6

Enter Product ID: 106

Enter Day (1-7): 6

Enter Units Sold: 45

Record 7

Enter Product ID: 107

Enter Day (1-7): 7

Enter Units Sold: 50

```
Output
Sales Records:
Product ID: 101 Day: 1 Units Sold: 20
Product ID: 102 Day: 2 Units Sold: 25
Product ID: 103 Day: 3 Units Sold: 30
Product ID: 104 Day: 4 Units Sold: 35
Product ID: 105 Day: 5 Units Sold: 40
Product ID: 106 Day: 6 Units Sold: 45
Product ID: 107 Day: 7 Units Sold: 50

Total Sales of Each Product:
Product 101 = 20 units
Product 102 = 25 units
Product 103 = 30 units
Product 104 = 35 units
Product 105 = 40 units
```

```
main.c
170
171     display();
172
173     totalSales();
174
175     highestSales();
176
177     printf("\nEnter Product ID to display weekly sales: ");
178     scanf("%d", &searchID);
179
180     productSales(searchID);
181
182     return 0;
183 }
184

Output
Product 106 = 45 units
Product 107 = 50 units

Highest Sales:
Product ID: 107
Day: 7
Units Sold: 50

Enter Product ID to display weekly sales: 101

Sales of Product 101:
Day 1 : 20 units

=== Code Execution Successful ===
```

2. Design a calculator program that evaluates postfix expressions using a stack data structure. The calculator should take a postfix expression as input and return the result of the expression. Given the postfix expression: 5 3 8 * + 4 - Implement the calculator program using a stack data structure to handle the postfix evaluation efficiently."

CODE:

```
#include <stdio.h>

#include <stdlib.h>

#include <ctype.h>

#include <string.h>


#define MAX 100


int stack[MAX];

int top = -1;


// Push operation

void push(int value)

{

    stack[++top] = value;

}


// Pop operation

int pop()

{

    return stack[top--];

}
```

```
int main()
{
    char exp[] = "5 3 8 * + 4 -";
    char *token;

    token = strtok(exp, " ");

    while (token != NULL)
    {
        if (isdigit(token[0]))
        {
            push(atoi(token));
        }
        else
        {
            int b = pop();
            int a = pop();

            switch (token[0])
            {
                case '+':
                    push(a + b);
                    break;
                case '-':
                    push(a - b);
                    break;
                case '*':
```

```

        push(a * b);

        break;

    case '/':

        push(a / b);

        break;

    }

}

token = strtok(NULL, " ");

}

printf("Postfix Expression: 5 3 8 * + 4 -\n");

printf("Result = %d\n", pop());

return 0;

}

```

INPUT & OUTPUT:

main.c	Output
<pre> 1 #include <stdio.h> 2 #include <stdlib.h> 3 #include <ctype.h> 4 #include <string.h> 5 6 #define MAX 100 7 8 int stack[MAX]; 9 int top = -1; 10 11 // Push operation 12 void push(int value) 13 { 14 stack[++top] = value; 15 } </pre>	<pre> Postfix Expression: 5 3 8 * + 4 - Result = 25 === Code Execution Successful === </pre>

Code of Conduct:

I Shaik Ayesha Tabassum(192525074) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:*S.Ayesha Tabassum*

RUBRICS FOR CALCULATION:

Numerical Problems

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation